Sisqual Import JSON

How SmarTaskUA converts a solved schedule into the partner's import format (v3)

When one of our Sisqual algorithms finishes solving a schedule, we convert it into a JSON document that Sisqual's platform can import directly. This document explains the current (v3) structure, what changed from the previous version, and walks through real examples end to end.

What changed in v3

The previous version gave each employee one OutRosterTeamDays entry **per team** worked that day. v3 changes where team information lives:

1. Team moved from day-level to task-level: The team an employee works (Storage / Checkout / Management / ...) is no longer on OutRosterTeamDays. It now lives inside **OutRosterTeamDayTasks**, one task per time segment, each with its own real TaskID + StartDate + EndDate.

2. TeamCode is now a fixed placeholder: TeamCode on OutRosterTeamDays is now a fixed placeholder (**"00"**) on every entry, since the real team assignment moved into the tasks.

3. Periods are grouped by time, not by team: Before, switching teams always started a new OutRosterTeamDays entry, even with no time gap. Now, periods are grouped by **contiguous time** across the whole day, regardless of team — a Checkout → Management switch with no gap between them stays one entry with two tasks, instead of two separate entries.

4. OutRosterTeamDayTasks is now populated: OutRosterTeamDayTasks is no longer always empty. Every worked segment of the day becomes one task, carrying the real calendar date (unlike OutScheduleUseds, which still uses a fixed placeholder date for its Start/EndDate fields).

Everything else — OutScheduleUseds as a deduplicated lookup table, the special DayType codes for day-off/vacation/etc., the 2-window cap per ScheduleCode, and the open questions about ScheduleWeight and real Sisqual codes — works the same way as before. Those are covered in full starting on the next page.

The two top-level pieces

The exported file always has exactly two arrays at the top level:

`OutRosterTeamDays`	One entry per employee, per day (usually just one, see the 2-window cap on page 5 for when a day needs more than one). Each entry lists the tasks worked that day and which ScheduleCode applies.
`OutScheduleUseds`	A lookup table of every distinct shift pattern referenced anywhere in OutRosterTeamDays — what time it starts, what time it ends, and how many minutes it's worth. Each entry is identified by a ScheduleCode.

Think of it like a spreadsheet split into two tabs: one tab says *"this employee, this day, these tasks, this shift-code"* for every workday; the other tab says *"this shift-code means these exact clock times."* Splitting it this way avoids repeating the same start/end times over and over for every employee who happens to work the same shift.

The overall shape looks like this:

```
{
  "OutRosterTeamDays": [ ... one entry per employee/day ... ],
  "OutScheduleUseds":  [ ... one entry per distinct shift pattern ... ]
}
```

1. OutRosterTeamDays

Each entry describes one employee's day. Instead of the team living directly on the entry, every worked segment of the day is listed as a task inside OutRosterTeamDayTasks — that's where the team (TaskID) and the real clock times actually live.

Field	Type	Meaning
`RosterCode`	string	Identifies which overall schedule/roster this entry belongs to. We use the schedule's title (e.g. the name you gave it when generating it).
`TeamCode`	string (fixed)	Always **"00"** — a fixed placeholder. Team assignment now lives at the task level, not the day level (see OutRosterTeamDayTasks below).
`EmployeeCode`	string	The employee's ID number, exactly as it appears in our system.
`Date`	date (YYYY-MM-DD)	The calendar date this entry applies to.
`ScheduleCode`	integer	Which shift pattern was worked. Look this number up in OutScheduleUseds (see section 2) to find the total worked minutes and clock-in/out windows.
`OutRosterTeamDayTasks`	array	One entry per worked time segment that day. Each task has a TaskID (the team worked, e.g. *Management*) plus its own real StartDate/EndDate.
`OutRosterTeamDayResponsibilities`	array	A place for named responsibilities (e.g. “key holder”). Always empty for now — explained in section 4.

Example: a split-team day

Employee **20072412** works Checkout from 10:00 to 11:00, then switches straight to Management until 14:00 — no gap in between, so it's one entry with two tasks:

```
{
  "RosterCode": "SISQUAL_OCTOBER_2025",
  "TeamCode": "00",
  "EmployeeCode": "20072412",
  "Date": "2025-10-01",
  "ScheduleCode": 6010840,
  "OutRosterTeamDayTasks": [
    {
      "TaskID": "Checkout",
      "StartDate": "2025-10-01T10:00:00",
      "EndDate": "2025-10-01T11:00:00"
    },
    {
      "TaskID": "Management",
      "StartDate": "2025-10-01T11:00:00",
      "EndDate": "2025-10-01T14:00:00"
    }
  ],
  "OutRosterTeamDayResponsibilities": []
}
```


Notice each task's StartDate/EndDate uses the **real** date (2025-10-01) — unlike OutScheduleUseds, whose Start/EndDate fields use a fixed placeholder date (explained in section 2).

Example: a day off

Days with no team assignment (day off, vacation, sick leave, ...) have no tasks at all:


```
{
  "RosterCode": "SISQUAL_OCTOBER_2025",
  "TeamCode": "00",
  "EmployeeCode": "20072412",
  "Date": "2025-10-05",
  "ScheduleCode": 2,
  "OutRosterTeamDayTasks": [],
  "OutRosterTeamDayResponsibilities": []
}
```


ScheduleCode 2 is a fixed special code ("Day Off") — see the full table of these in section 2.

2. OutScheduleUseds

This is the lookup table. Every ScheduleCode used anywhere in OutRosterTeamDays has exactly one matching entry here that spells out what that code actually means in clock time. Unlike the tasks (which are grouped by *team*), the periods here are grouped purely by **contiguous time** — a team switch with no time gap doesn't start a new period, but a real gap (like an unpaid break) does.

Field	Type	Meaning
`ScheduleCode`	integer	The same code referenced from OutRosterTeamDays. Matches exactly one entry here.
`DayType`	integer	What kind of day this is: **0** = a real worked shift, and small numbers above that mean day-off, vacation, sick leave, etc. (full list below).
`ScheduleWeight`	integer (minutes)	How many minutes the shift is worth. For a worked shift, this is the total time worked. For day-off/vacation/etc. it's always 0.
`StartDate1 / EndDate1`	datetime, optional	Clock-in and clock-out time of the first contiguous work period of the day.
`StartDate2 / EndDate2`	datetime, optional	Clock-in and clock-out time of a second contiguous period, if there's a real time gap somewhere in the day (e.g. a lunch break).

Example: what ScheduleCode 6010840 actually means

Continuing the split-team example from section 1 — Checkout then Management with no gap between them — the whole day is **one continuous period**, 10:00 to 14:00:


```
{
  "ScheduleCode": 6010840,
  "DayType": 0,
  "ScheduleWeight": 240,
  "StartDate1": "2026-01-01T10:00:00",
  "EndDate1": "2026-01-01T14:00:00"
}
```


240 minutes = 4 hours, matching the full 10:00–14:00 span (both tasks combined). The date part of StartDate1/EndDate1 (2026-01-01) is just a fixed placeholder — only the time-of-day (10:00, 14:00) actually matters here; the real date lives on the OutRosterTeamDays / task side.

Example: a real gap in the day (split shift)

When there's an actual time gap — not just a team switch — a second period appears. Storage 08:00–10:00, then Checkout 12:00–14:00 (a 2-hour gap in between):

```
{
  "ScheduleCode": 4810840,
  "DayType": 0,
  "ScheduleWeight": 240,
  "StartDate1": "2026-01-01T08:00:00",
  "EndDate1": "2026-01-01T10:00:00",
  "StartDate2": "2026-01-01T12:00:00",
  "EndDate2": "2026-01-01T14:00:00"
}
```


This means: work 08:00–10:00 (2 hours), then again 12:00–14:00 (2 hours), totalling 240 minutes (4 hours) — even though 6 hours pass between clock-in and final clock-out.

Special codes for non-worked days

A handful of fixed, low ScheduleCodes are reserved for days where nothing was actually worked. These always have ScheduleWeight 0 and no Start/EndDate fields:

Code	Meaning	Description
2	Day Off	A regular day off. Can be swapped with another day if needed.
3	Forced Day Off	A day off that cannot be changed or swapped.
4	Vacation	Employee is on vacation — cannot work.
5	Unavailable	Employee marked unavailable for other reasons.
6	Medical Leave	Employee is on medical/sick leave.
7	Store Closed	The store itself is closed that day (holiday, etc.).
8	Left Off / Swapped Off	A normally-scheduled work day that ended up unworked.
9	Unassigned	A work day that, unexpectedly, got no shift assigned — a signal something needs a human look.

3. A complete example, start to finish

Here is one employee's real two-day slice, showing both parts of the file together. Employee **20072412** works Checkout then Management (no gap) on October 1st, then has a day off on October 5th.


```
{
  "OutRosterTeamDays": [
    {
      "RosterCode": "SISQUAL_OCTOBER_2025",
      "TeamCode": "00",
      "EmployeeCode": "20072412",
      "Date": "2025-10-01",
      "ScheduleCode": 6010840,
      "OutRosterTeamDayTasks": [
        {
          "TaskID": "Checkout",
          "StartDate": "2025-10-01T10:00:00",
          "EndDate": "2025-10-01T11:00:00"
        },
        {
          "TaskID": "Management",
          "StartDate": "2025-10-01T11:00:00",
          "EndDate": "2025-10-01T14:00:00"
        }
      ],
      "OutRosterTeamDayResponsibilities": []
    },
    {
      "RosterCode": "SISQUAL_OCTOBER_2025",
      "TeamCode": "00",
      "EmployeeCode": "20072412",
      "Date": "2025-10-05",
      "ScheduleCode": 2,
      "OutRosterTeamDayTasks": [],
      "OutRosterTeamDayResponsibilities": []
    }
  ],
  "OutScheduleUseds": [
    {
      "ScheduleCode": 6010840,
      "DayType": 0,
      "ScheduleWeight": 240,
      "StartDate1": "2026-01-01T10:00:00",
      "EndDate1": "2026-01-01T14:00:00"
    },
    {
      "ScheduleCode": 2,
      "DayType": 1,
      "ScheduleWeight": 0
    }
  ]
}
```


Reading it left to right:

1. October 1st entry says: employee 20072412, ScheduleCode 6010840, with two tasks — Checkout 10:00–11:00, then Management 11:00–14:00.

2. Look up ScheduleCode 6010840 in OutScheduleUseds: DayType 0 (a real worked day), one continuous period 10:00–14:00, worth 240 minutes (4 hours) — matching both tasks combined.

3. October 5th entry says: employee 20072412, no tasks, ScheduleCode 2.

4. Look up ScheduleCode 2 in OutScheduleUseds: DayType 1 (Day Off), worth 0 minutes, no clock times — because nothing was worked.

When a day needs more than one entry

OutScheduleUseds can only hold 2 time windows per ScheduleCode. If a day has more than 2 real time gaps, it can't fit in one entry — so it splits into multiple OutRosterTeamDays entries for the same employee and date, each with its own ScheduleCode and only the tasks that fall within its time window. No worked time is ever lost this way, it just spreads across more than one entry.

4. OutRosterTeamDayResponsibilities is still always empty

Unlike OutRosterTeamDayTasks (now populated, see section 1), **OutRosterTeamDayResponsibilities** is still always an empty array (**[]**) in every entry we produce. Our scheduling model has no concept of named responsibilities within a shift (e.g. “key holder for the day”, “cash reconciliation”) — only which team an employee works, which is now captured via tasks.

Populating this would require a real extension to our data model (a new responsibility concept in problem.json, plus new decision variables in the solvers themselves) — not just a change to the export step.

> **OPEN QUESTION:** The spec marks this field as optional, so an empty array should be a valid import. Worth confirming directly with Sisqual whether they actually require non-empty responsibility data for a schedule to import successfully.

5. Other open questions

A couple of other things in this export are our best interpretation of the partner spec, not confirmed facts — worth keeping in mind:

ScheduleWeight and unpaid breaks. The partner's own example file shows ScheduleWeight values slightly lower than the raw time between clock-in and clock-out in every single example — suggesting some kind of standard break deduction. We couldn't confirm the exact rule, so our export currently reports the full worked time with no deduction.

ScheduleCode numbers are not Sisqual's real codes. Sisqual's platform likely has its own registry of numeric codes for known shift patterns. We don't have access to that registry, so the codes in this export are generated by us — consistent and reproducible from one export to the next, but not guaranteed to match anything already defined on Sisqual's side.

TeamCode is now a placeholder. Since v3 moves team information into tasks, TeamCode on OutRosterTeamDays no longer carries real information (always "00"). Worth confirming with Sisqual that this is an acceptable placeholder value for their import, since the field is documented as required rather than optional.